# Engineering Local Agentic Coding Harnesses with Orthogonalized Small Language Models

Systems Architecture, Representation Geometry,
and Airgapped Security Analysis

Alexi Vieni*
Independent Researcher, Canada
alexivieni00@gmail.com

Technical Report · Draft for Review · November 2026

**Abstract**

Agentic coding systems put a language model in a loop: retrieve context, propose an edit, run checks, try again. This report engineers that loop to run entirely on local, airgapped hardware from three components: small language models orthogonalized against refusal, a deterministic client-side harness, and a quantitative design model for the machines they run on. Refusal is mediated by a low-dimensional subspace of the residual stream; projecting the weights onto the orthogonal complement of the refusal direction, $P = I - \hat{r}\hat{r}^\top$, removes the false-positive refusals that halt unattended pipelines without retraining. The harness offloads memory, retrieval and verification from the weights, so that code, not the model, decides what is kept. The design model treats GPU memory, RAM, storage and the network as one memory hierarchy, and shows that the expected time per accepted change is the call time divided by an attempt-weighted pass rate, $C = t/\bar{p}$; from these follow rules for model size, context window, swaps, batching, multi-machine splits and role placement. Each element is mapped to a working prototype, and an evaluation on a 14B and a 32B model is specified. The model proposes; code decides.

**Keywords:** subspace orthogonalization, representation geometry, refusal direction, small language models, agentic coding, local inference, memory hierarchy, KV cache, batching, speculative decoding, cost model

## 1 Introduction and Operational Intent

Agentic coding systems put a language model in a loop: they retrieve context, propose an edit, run checks, and try again until the checks pass [5, 20, 22]. Relying on public cloud endpoints for that loop carries risks that matter for sensitive work: data leaves the machine, third parties log it, and persistent false-positive refusal filters break automated pipelines whenever ordinary technical artifacts trip rigid safety boundaries. Local operation removes the first two, has no per-token costs or rate limits, and leaves every part of the stack open to inspection and change.

To that end, the architecture establishes an entirely local, airgapped sandbox stack. It combines a small language model in the 4B to 32B parameter class, latent-space weight orthogonalization, and a deterministic client-side agentic harness. Orthogonalization removes refusal behavior from the weights (Section 2). The harness offloads memory, file traversal, indexing and verification out of the weights, turning the small language model into a fast, unhindered execution engine whose work is accepted by code rather than by the model itself.

*The author is a Security Supervisor in the Canadian Armed Forces, with operational service in 2024. This is an independent, personal project; the views expressed are the author's own and do not represent the Department of National Defence or the Canadian Armed Forces.

On consumer hardware—one GPU with 6–8 GB of memory, 16–32 GB of RAM, perhaps a second machine—that engine has hard limits. The model sees a few thousand tokens of a much larger repository, generates tens of tokens per second at best—far fewer once its layers spill into system RAM, and under one per second once they spill to storage—loses the thread across files, and repeats itself when stuck. This report argues that on such hardware the harness, not the model, is the system, and that its design follows from a few measurable quantities rather than from intuition: where a model's bytes live, what a call costs, and how often its output passes. It makes five contributions.

(i) **Representation geometry and subspace orthogonalization** (Section 2): a synthesis of the evidence that refusal is mediated by a low-dimensional subspace of the residual stream [2, 21], applied as a deployment step for unattended local agents—the rank-1 projection that removes it from the weights without retraining, its reported costs [23], and a gate for deploying it.

(ii) **A hardware model** (Section 4): memory fit with the KV cache; a decoding bound over the GPU, RAM and storage tiers, including mixture-of-experts models; prompt processing; model swaps and the page-cache condition that keeps them cheap; batching; roles and layers on several machines; and the choice of model size, precision and window.

(iii) **A loop model** (Section 5): the expected time and energy per accepted change, a role-placement inequality that includes cross-role effects, when feedback retries dominate sampling and how batching changes the comparison, and a terminating loop-detection rule.

(iv) **An implementation map** (Section 8) tying each element to working code, including unattended operation and gated self-improvement (Section 7).

(v) **A pre-specified evaluation** (Section 9) that tests these predictions on a 14B and a 32B model, base and orthogonalized.

# 2 Mathematical Foundations: Representation Geometry and Subspace Orthogonalization

The foundational mathematical mechanism relies on representation engineering and linear vector subspace interventions in the transformer's latent activation space. Its purpose in this architecture is operational: an unattended loop fails on a refusal as surely as on a crash. Over-refusal—declining a safe request because its wording resembles an unsafe one—is a documented behavior of safety-tuned models [17], and technical artifacts trigger it readily: an exploit write-up, a socket hook, shellcode inside a packet capture, a detection rule that must quote what it detects. Because of how refusal is represented, it can be removed without retraining.

## 2.1 The refusal subspace

Research established that instruction tuning does not distribute safety rules randomly across a neural network. Instead, refusal behavior is overwhelmingly mediated by a low-dimensional subspace in the residual stream $x \in \mathbb{R}^d$, defined by a normalized unit direction vector $\hat{r} \in \mathbb{R}^d$: prompts the model will refuse push activations along it, adding it makes the model refuse harmless prompts, and removing it stops refusal [2]. When sensitive keywords such as *exploit*, *socket hook* or *shellcode* enter the context, transformer activations accumulate significant projection along $\hat{r}$, steering output probability distributions toward refusal templates. The direction is estimated as a difference of

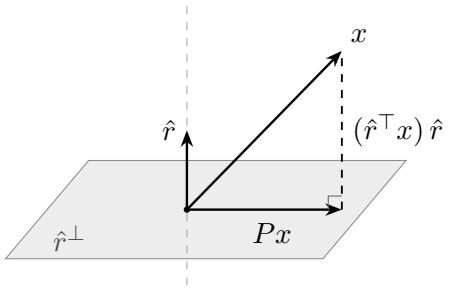



**Figure 1:** Rank-1 orthogonalization. The component of an activation $x$ along the refusal direction $\hat{r}$ (dashed) is removed; the remainder $Px = x - (\hat{r}^\top x)\,\hat{r}$ lies in the orthogonal complement $\hat{r}^\perp$ and passes through unchanged.

means between activations on refused and complied prompts at one layer and token position:

$$\mathbf{r} = \mu_{\text{refused}} - \mu_{\text{complied}}, \qquad \hat{r} = \frac{\mathbf{r}}{\|\mathbf{r}\|}. \tag{1}$$

Later work sharpens the picture rather than overturning it. Refusal is carried by several independent directions and multi-dimensional *concept cones*, and two orthogonal directions can still interact when one is intervened on [21]. Low-dimensional is the robust finding; "exactly one direction" is the useful approximation.

## 2.2 Rank-1 orthogonal projection matrix

To neutralize this refusal mechanism without expensive model retraining, a projection matrix $P$—a rank-1 update of the identity—is constructed to project activation vectors onto the orthogonal complement of the refusal subspace spanned by $\hat{r}$:

$$P = I - \hat{r}\hat{r}^\top. \tag{2}$$

Applying this linear transformation directly to layer weight matrices, $W' = PW$, or to streaming activations guarantees that for any activation vector $x$, its inner product with the refusal direction is strictly zero, because $\hat{r}^\top\hat{r} = 1$:

$$\langle Px, \hat{r}\rangle = \hat{r}^\top(I - \hat{r}\hat{r}^\top)x = (\hat{r}^\top - \hat{r}^\top)x = 0. \tag{3}$$

Applied once to every matrix that *writes* into the residual stream—the token embedding and each layer's attention and MLP output projections—the edit leaves the model unable to write along $\hat{r}$ at any depth (Figure 1). By forcing the projection along the refusal vector to zero, activation drift toward refusal along $\hat{r}$ is eliminated at every layer. This restricts the residual stream to the hyperplane $\hat{r}^\perp$; whatever a layer writes within that hyperplane passes through unchanged. Nothing is retrained and nothing is added at inference; projecting activations at run time is equivalent but costs a hook per layer.

## 2.3 What orthogonalization costs

- **It adds no knowledge and no speed.** Capability comes from the base model; throughput comes from its size, its quantization and where its bytes live (Section 4).
- **It is not free.** In a comparison of four abliteration tools, the best ones moved MMLU and HellaSwag by tenths of a point, while GSM8K ranged from +1.5 to −18.8 points depending on tool and model; mathematical reasoning was the most sensitive [23]. An abliterated 8B model lost quality that preference tuning then largely recovered [12].

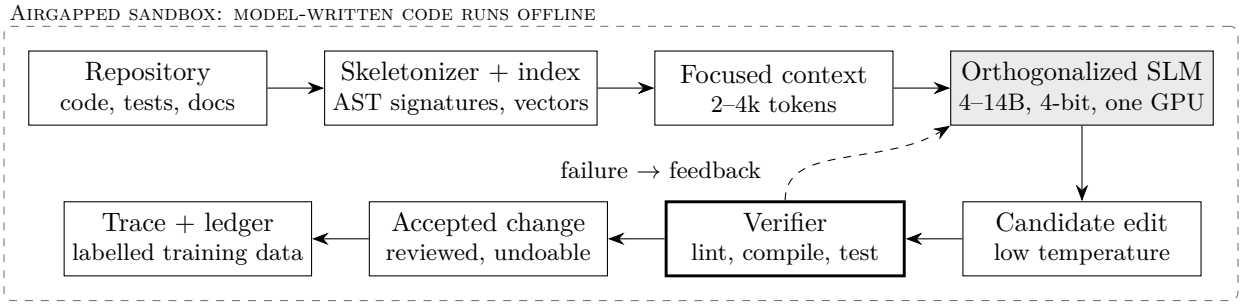



**Figure 2:** The harness loop. The model sees only a focused slice of the repository and only proposes; the verifier's exit code, not the model's own assessment, decides what is kept. Every call is traced and later labelled by that verdict.

- **It does not remove hedging.** Practitioners using community abliteration tools report that refusals disappear while disclaimers and stylistic hedging often persist, which suggests they are not carried by the refusal direction; removing them requires deeper intervention (Section 10).
- **It removes caution along with refusal.** What the model no longer checks, the harness has to (Section 7).

### 2.4 Deployment gate

The same task suite should be run on the base and the orthogonalized model, and the edit kept only if over-refusal on the intended workload falls *and* pass rates hold—reasoning-heavy tasks first, since they degrade first. Section 9 specifies such a test (H1). Healing by preference tuning [16] can draw on the harness's own traces: every step that failed and then passed yields a rejected and an accepted reply to the same prompt, labelled by the verifier rather than by a person.

## 3 End-to-End System Pipeline and Harness Architecture

The local airgapped pipeline orchestrates four core decoupled subsystems arranged in one loop (Figure 2). Everything that needs memory, judgment about correctness, or access to the world lives outside the model.

**Airgapped sandbox environment.** Isolates all operations, codebases, indexing engines and model runtimes offline, with no telemetry leaving the machine. Code the model writes runs under an explicit network policy, offline by default (Section 7).

**AST symbol skeletonizer.** Uses AST parsing to extract imports, class signatures and function headers for the whole repository while stripping unused boilerplate, and includes the functions relevant to the request in full (Section 5.5). This compresses context from the size of the repository—up to millions of raw tokens—to two to four thousand targeted tokens, preserving the small model's attention for what matters.

**Orthogonalized SLM engine.** Dense 4–14B models at 4-bit on one GPU, with orthogonalized weight projections across residual layers (Section 2), generate without policy halts; larger models take upstream roles where the cost model says they pay (Section 5.2).

**Best-of-$N$ candidate sampler and verifier.** Draws candidate diffs at low temperature—$N$ at once when their KV caches fit (Section 5.3)—and routes them through local linters, compilers and tests, establishing ground truth by execution rather than by soft model self-correction. Failure output is fed back to the next attempt.

# 4 Hardware Model

The design choices in Section 3 follow from a small set of quantities. This section models the machine: where a model's bytes live, how fast each tier delivers them, and what moving them costs. Section 5 models the loop that runs on it. Both models are deliberately simple; their purpose is to make each design decision checkable against measurements, and every quantity in them is observable in the harness's traces or the system's counters.

## 4.1 Memory fit

A transformer with $L$ layers, $h_{\mathrm{kv}}$ key–value heads of dimension $d_h$, and $b$ bytes per cached element stores

$$\kappa = 2\, L\, h_{\mathrm{kv}}\, d_h\, b \tag{4}$$

bytes of KV cache per token (keys and values). A model whose weights occupy $M$ bytes runs entirely on a GPU with $V$ bytes of memory, at a context window of $n$ tokens and runtime overhead $O$, if and only if

$$M + \kappa\, n + O \;\le\; V, \qquad \text{equivalently} \qquad n \;\le\; n^{*} = \left\lfloor \frac{V - O - M}{\kappa} \right\rfloor. \tag{5}$$

For example, a 7B model with $L = 28$, $h_{\mathrm{kv}} = 4$ and $d_h = 128$ needs $\kappa = 56\,\mathrm{KiB}$ per token at 16-bit precision ($b = 2$), or 0.88 GiB for a 16k window; an 8-bit cache (`q8_0`, $b = 34/32$) halves this to 0.47 GiB. Equation (5) is why the cache type and the window, not only the weights, decide whether a model fits.

## 4.2 The memory hierarchy: GPU, RAM and storage

Decoding one sequence is memory-bandwidth bound: each generated token reads every weight once, and every entry of its KV cache once [15]. A model's bytes may live in three tiers—GPU memory, system RAM, and storage reached through memory-mapped files—whose bandwidths span nearly two orders of magnitude: roughly 200–350 GB/s for a laptop GPU, 50–90 GB/s for dual-channel DDR4 or DDR5, and 3.5–7 GB/s for an NVMe drive. If $D_\tau$ bytes of weights and cache are read from tier $\tau$ for each token, with $\bar{n} \le n$ tokens in context, then

$$t_{\mathrm{tok}} \;\approx\; \sum_{\tau \,\in\, \{\mathrm{gpu},\, \mathrm{ram},\, \mathrm{disk}\}} \frac{D_\tau}{B_\tau}, \qquad \sum_{\tau} D_\tau \;=\; M + \kappa\, \bar{n}. \tag{6}$$

Three consequences follow.

**GPU and RAM.** With a fraction $g$ of the layers on the GPU, the rest in RAM, and the cache traffic ignored, Equation (6) reduces to

$$t_{\mathrm{tok}} \;\approx\; \frac{g\, M}{B_{\mathrm{gpu}}} + \frac{(1-g)\, M}{B_{\mathrm{ram}}}. \tag{7}$$

The RAM term dominates quickly: below $g \approx 0.86$ it is the larger of the two, and losing the last tenth of a model to RAM costs a third of its speed. A 14B model ($M \approx 9\,\mathrm{GB}$) on a 6 GB card keeps about half of its weights on the GPU at an 8k window and is bounded near 9 tokens/s. A 32B model ($M \approx 20\,\mathrm{GB}$) with a quarter of its layers on the GPU is bounded at about 3 tokens/s, against about 15 tokens/s were it to fit.

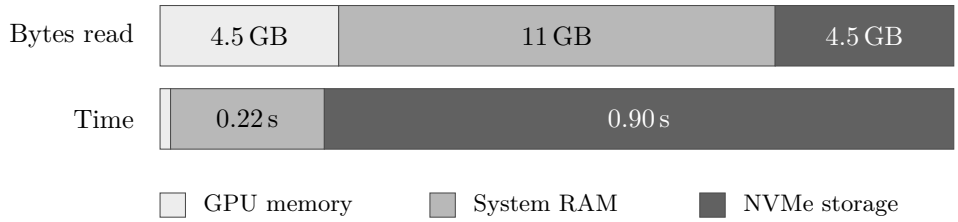



**Figure 3:** Where a 32B model's bytes are read from, and where its decoding time goes, on a machine with a 6 GB GPU, 16 GB of RAM and an NVMe drive (Equation (6) with $B_{\text{gpu}} = 300$, $B_{\text{ram}} = 50$ and $B_{\text{disk}} = 5$ GB/s). Storage supplies a fifth of the bytes and takes four fifths of the 1.1 s per token; the GPU's 0.015 s is the sliver at the left.

**The cache is read too.** The cache term $\kappa\bar{n}$ grows with the context. For a 14B model with $L = 48$, $h_{\text{kv}} = 8$ and $d_h = 128$ at 16-bit precision, $\kappa = 192$ KiB per token, so a full 16k window adds 3 GiB of reads to every generated token—about a third of the weight traffic. Long contexts slow every token, not only the prompt; an 8-bit cache halves the term.

**Storage as a tier.** A model larger than GPU memory and RAM together still runs if its weights are memory-mapped: the operating system pages the remainder from storage on every token, and $B_{\text{disk}}$ enters Equation (6). For a 32B model on a 6 GB GPU with 16 GB of RAM, of which about 11 GB remain after the operating system and runtime, roughly 4.5 GB stay on the GPU, 11 GB in RAM and 4.5 GB on an NVMe drive. With $B_{\text{disk}} = 5$ GB/s, $t_{\text{tok}} \approx 4.5/300 + 11/50 + 4.5/5 \approx 1.1$ s: under one token per second, four fifths of it spent reading the fifth of the model that lives on storage (Figure 3). With 32 GB of RAM the storage term disappears and the bound rises to about 3 tokens/s. The estimate assumes the operating system keeps the same 11 GB resident; a least-recently-used cache facing a cyclic scan larger than itself can evict every page before its reuse, and random reads are slower than sequential ones, so real figures are lower still.

**Mixture-of-experts models.** Equation (6) counts the bytes a token reads, not the bytes a model holds. A mixture-of-experts model routes each token to a few of its experts [19], so $D_\tau$ counts only the shared weights and the active experts, while the fit conditions still count every expert. A model with 30B parameters of which about 3B are active needs some 18 GB at 4-bit but reads only about 2 GB per token: with its experts in RAM, Equation (6) bounds it near 25 tokens/s, against under 3 for a dense 30B model in RAM. Placement within the hierarchy matters: a runtime that splits by whole layers sends attention weights, which every token reads, to RAM along with the experts, while keeping only the expert tensors in RAM leaves the shared weights on the GPU. When part of the model pages from storage, each token reads only the experts it uses, so in expectation the storage term shrinks in the same proportion.

### 4.3 Prompt processing

Prompt processing is the opposite regime. All prompt tokens pass through each layer together, so the weights are read once per batch of tokens and the cost is arithmetic. A forward pass costs about $2|\theta|$ floating-point operations per token for a model with $|\theta|$ parameters (attention adds a term that grows with the context), so with peak arithmetic throughput $\Pi$ and utilization $\eta$ the prompt-processing rate is approximately

$$r_p \;\approx\; \frac{\eta\,\Pi}{2\,|\theta|}. \tag{8}$$

A 14B model needs about 28 GFLOP per prompt token. At an illustrative effective 20 TFLOP/s that is about 700 tokens/s, so an uncached 8k-token prompt costs some 12 s before the first generated token. Layers held in RAM are either computed by the CPU, at a fraction of the GPU's rate, or streamed to the GPU for each batch of prompt tokens at a cost of $(1-g)M/B_{\text{pcie}}$, which larger batches amortize. This is why the cached fraction of the prompt (Section 5.1) matters as much as the decoding bound.

### 4.4 Loading and swapping models

When the loop moves between two models that do not fit on the GPU together, the outgoing model is unloaded and the incoming one loaded. Its weights come from the operating system's page cache if they are still in RAM, and from storage otherwise, and then cross the PCIe bus:

$$\lambda \;\approx\; \lambda_0 + \frac{M}{B_{\text{pcie}}} + \mathbf{1}[\text{not cached}]\,\frac{M}{B_{\text{disk}}}, \tag{9}$$

where $\lambda_0$ covers allocation and setup. At $B_{\text{pcie}} \approx 12$–$25$ GB/s the bus adds under a second for a 9 GB model; storage adds more. Swaps stay warm only while every model in the rotation fits in the page cache together:

$$\sum_{m \,\in\, \text{rotation}} M_m \;\le\; R - R_0, \tag{10}$$

with $R$ the system RAM and $R_0$ what the operating system, the runtime and other programs hold. With 16 GB of RAM ($R - R_0 \approx 11$–$12$ GB), a 14B builder and a 4B router ($\approx 11.5$ GB) only just stay warm; a 14B and a 32B model ($\approx 29$ GB) cannot, and every swap re-reads the incoming model from storage—1.3 to 2.6 s for 9 GB at NVMe speeds, before setup. Inequality (10) is the RAM tier's role in the loop: it decides whether using several models is cheap or expensive.

### 4.5 Batching

A GPU that decodes several sequences at once reads the weights once per step for all of them, and each sequence's cache once. For a model held entirely on the GPU,

$$t_{\text{step}}(N) \;\approx\; \frac{M + N\,\kappa\,\bar{n}}{B_{\text{gpu}}}, \qquad N \;\le\; N^{*} = \left\lfloor \frac{V - O - M}{\kappa\, n} \right\rfloor, \tag{11}$$

where $\bar{n}$ is the mean context length during decoding and $N^{*}$ the number of caches that fit. While $N \le N^{*}$, $N$ sequences cost little more than one. A 7B model at 4-bit ($M \approx 4.7$ GB) on a 6 GB card, with an 8-bit cache and a 4k window, has room for $N^{*} = 4$ caches, and four sequences decode in about 1.08 times the time of one; at a 16k window not even two fit. The window therefore trades context against parallel candidates—a trade the loop model prices in Section 5.3.

### 4.6 Several machines

A second machine adds memory and a second set of bandwidths. It can be used in two ways.

**Roles on different machines.** Only text crosses the network—prompts and replies, kilobytes per call—so the network itself costs almost nothing. Each machine keeps its own models resident, with the prefix caches of the roles it serves, so the swaps between those roles disappear. And calls that do not depend on each other can overlap: if the judge's verdict on step $s$ is not needed to start step $s+1$, running builder and judge on different machines replaces a sum by a maximum,

$$\sum_{s} \max(T_{b,s},\, T_{j,s}) \;\le\; \sum_{s} (T_{b,s} + T_{j,s}), \tag{12}$$

where $T_{b,s}$ and $T_{j,s}$ are the builder's and the judge's time on step $s$; the left side is the run time with the two overlapped, the right side without. The price is redoing step $s+1$ whenever the judge rejects step $s$.

**Layers on different machines.** Splitting one model's layers across machines lets a model run that fits on no single machine, but one sequence still visits every layer in turn:

$$t_{\text{tok}} \;\approx\; \sum_i t_{\text{tok},i} \;+\; \nu\,\delta_{\text{net}}, \tag{13}$$

where $t_{\text{tok},i}$ is Equation (6) applied to the layers on machine $i$, and each of the $\nu$ network hops per token adds a latency $\delta_{\text{net}}$. The activation passed at each hop is $d\,b$ bytes per token—10 KiB for a model width $d = 5120$ at 16-bit—so even an 8k-token prompt moves only 80 MiB, and a local network's sub-millisecond latency is small against the tens to hundreds of milliseconds a token takes. Splitting adds capacity, not parallelism, but capacity is what a small machine lacks. Two machines with a 6 GB GPU and 16 GB of RAM can each hold 4.5 GB of the 32B model on the GPU and 5.5 GB in RAM; Equation (13) then gives $t_{\text{tok}} \approx 2\,(4.5/300 + 5.5/50) \approx 0.25$ s, about 4 tokens/s, against under one when a single machine pages from storage.

### 4.7 Choosing size, precision and window

The quantities above combine into one choice. The loop's output is accepted changes per second, $\Phi = 1/C = \bar{p}/t$ (Section 5.1), so the model size $|\theta|$, the precision $q$ in bits per weight (with $M = |\theta|\,q/8$) and the window $n$ should solve

$$\max_{|\theta|,\,q,\,n} \;\frac{\bar{p}(|\theta|, q, n)}{t(|\theta|, q, n)} \tag{14}$$

subject to the memory constraints of Sections 4.1–4.4. The time $t$ follows from Equations (6)–(8); the pass rate $\bar{p}$ must be measured. Two regularities keep the search small. Decoding time is linear in bits, and for a fixed memory budget 4-bit weights give almost universally the best zero-shot accuracy [9], so $q = 4$ is the default and the search runs over size and window. And because a window that pushes layers off the GPU costs speed hyperbolically (Section 4.2), $n$ should stop at $n^*$ unless the pass rate gains more than the time loses. Table 1 collects the rules that follow from this section.

## 5 Loop Model

Given the machine, the loop's behavior is set by three further quantities: the time a call takes, the probability that its output passes the verifier, and how often each role is called.

### 5.1 Cost of an accepted change

A call with $n_{\text{in}}$ prompt tokens and $n_{\text{out}}$ generated tokens takes

$$t \;=\; \frac{(1-\rho)\,n_{\text{in}}}{r_p} + n_{\text{out}}\,t_{\text{tok}}, \tag{15}$$

with $r_p$ from Equation (8), $t_{\text{tok}}$ from Equation (6), and $\rho$ the fraction of the prompt served from a cached prefix. Keeping the stable part of every prompt first (instructions, project context, goal) and the per-attempt part last (feedback, failure output) makes $\rho$ close to one on retries: an 8k-token context followed by 300 tokens of feedback gives $\rho \approx 0.96$.

**Table 1:** The hardware model in brief: each resource, what it governs, and the rule that follows. Typical values are for a current laptop.

| Resource | Typical | Governs | Rule |
|---|---|---|---|
| GPU memory | 6–8 GB at 200–350 GB/s | Fit, decoding speed and parallel candidates; Eqs. (5), (7), (11) | Keep the builder entirely on the GPU; stop the window at $n^*$; use an 8-bit KV cache. |
| GPU compute | ∼20 TFLOP/s effective | Prompt processing; Eq. (8) | Put stable text first so that retries reuse the cached prefix; prune agent history. |
| System RAM | 16–32 GB at 50–90 GB/s | Spilled layers and warm swaps; Eqs. (7), (10) | Keep the model rotation within $R - R_0$; hold one large model at a time; for weights that must live in RAM, prefer a mixture-of-experts model. |
| Storage | NVMe at 3.5–7 GB/s | Cold loads and paged weights; Eqs. (6), (9) | Never page weights during a loop; warm the page cache before a run. |
| PCIe bus | 12–25 GB/s | Loads and streamed prompt batches; Eqs. (8), (9) | Fewer swaps; large prompt batches when layers stream from RAM. |
| Network | 1 Gb/s, under 1 ms | Roles or layers on other machines; Eqs. (12), (13) | Route whole roles to other machines; split a model's layers only when it would otherwise page from storage. |

Let each attempt at a step pass the verifier with probability $p$, and let the harness allow at most $k$ attempts. The expected number of attempts is $\sum_{i=0}^{k-1}(1-p)^i = \big(1-(1-p)^k\big)/p$, and the step succeeds with probability $1-(1-p)^k$. The expected time per *accepted* change is therefore

$$C \;=\; \frac{t\,\big(1-(1-p)^k\big)/p}{1-(1-p)^k} \;=\; \frac{t}{p}, \tag{16}$$

independent of the attempt cap $k$; the cap bounds the worst case, not the average.

Feedback changes the pass probability from one attempt to the next. Let attempt $i$ pass with probability $p_i$, and let $Q_i = \prod_{j\le i}(1-p_j)$, with $Q_0 = 1$, be the probability that the first $i$ attempts all fail. Attempt $i$ is made with probability $Q_{i-1}$ and succeeds with probability $Q_{i-1}p_i$, so

$$C \;=\; \frac{t\sum_{i=1}^{k} Q_{i-1}}{\sum_{i=1}^{k} Q_{i-1}\,p_i} \;=\; \frac{t}{\bar{p}}, \qquad \bar{p} \;=\; \frac{\sum_{i=1}^{k} Q_{i-1}\,p_i}{\sum_{i=1}^{k} Q_{i-1}}\,: \tag{17}$$

the time per accepted change is the call time divided by the attempt-weighted mean pass rate. Equation (16) is the special case $p_i = p$. If feedback never makes the model worse ($p_i \ge p_1$), then $\bar{p} \ge p_1$ and $C \le t/p_1$: the first-attempt pass rate gives an upper bound on the cost, and the gap between that bound and the measured $C$ is a direct measure of the value of feedback.

Equation (17) is the quantity a model should be judged by. A model twice as fast that passes half as often costs the same; if it passes less often than that, it is the slower model in practice. The same form gives energy: at an average power draw $\mathcal{P}$ during calls, the energy per accepted change is $E = \mathcal{P}\,C$, the quantity to minimize on a laptop running on battery. The harness reports $C$ per role and model from its own runs, and its battery mode trades pass rate for energy: on battery it shortens the window and the time models stay loaded, drops sampled candidates, and judges only the final result.

### 5.2 Placing roles across models

A run needs about $c_r$ accepted outputs from each role $r$—a plan or two, tens of builds. With a model assigned to each role—giving role $r$ a call time $t_r$ and acceptance probability $p_r$—and load time $\lambda$ (Equation (9)) for each of the $S$ model swaps the assignment causes on one GPU, the expected run time is

$$J \;=\; \sum_r c_r \,\frac{t_r}{p_r} \;+\; \lambda\, S. \tag{18}$$

Placement has two effects. The *direct* effect of moving role $r$ from a small model $s$ to a large model $\ell$ is $c_r(t_{\ell,r}/p_{\ell,r} - t_{s,r}/p_{s,r})$. Large models are slower per call, so this term is usually a cost: it is negative only if the large model's acceptance rate exceeds the small one's by more than its slowdown. The *indirect* effect runs through the roles that consume a role's output. A plan is consumed by every builder call; if a better plan raises the builder's acceptance probability from $p_b$ to $p_b'$, the builder's term falls by $c_b\, t_b\, (1/p_b - 1/p_b')$. Moving the planner to the large model therefore pays when

$$c_b\, t_b \left(\frac{1}{p_b} - \frac{1}{p_b'}\right) \;>\; c_r \left(\frac{t_{\ell,r}}{p_{\ell,r}} - \frac{t_{s,r}}{p_{s,r}}\right) + \lambda\, \Delta S. \tag{19}$$

Because a run needs tens of accepted builds and one to three plans, a modest gain in the builder's acceptance rate can pay for a much slower planner. Applied to the builder itself, which has no downstream consumer, the same inequality keeps it on the fast model. On two machines the swap term vanishes, and calls that do not depend on each other leave the critical path (Inequality (12)).

**Example.** Take illustrative values: $c_b = 20$ builds with calls of $t_b = 15\,\text{s}$; one plan, whose call takes 30 s and is accepted with probability 0.8 on the 14B model, or 120 s and 0.9 on the 32B model; and $\lambda\,\Delta S = 15\,\text{s}$ for the swaps to the 32B model and back. The right-hand side of (19) is $(133.3 - 37.5) + 15 \approx 111\,\text{s}$. If the 32B model's plans raise the builder's acceptance rate from 0.40 to 0.50, the left-hand side is $300\,(2.5 - 2.0) = 150\,\text{s}$ and the move pays; if they raise it only to 0.45, the left-hand side is 83 s and it does not. A ten-point gain in builder acceptance pays for a planner four times slower; a five-point gain does not. Which side wins is an empirical question, and Section 9 tests it.

### 5.3 Retries, sampling and batching

Each attempt can draw one candidate or several. Without batching—when $N$ caches do not fit, $N > N^*$ in Equation (11)—$N$ independent samples cost $Nt$, the same as $N$ sequential attempts. Independent samples succeed with probability $1 - (1 - p_1)^N$. Sequential attempts that receive the verifier's failure output have per-attempt probabilities $p_1, p_2, \ldots$, and succeed with probability

$$1 - \prod_{i=1}^{N} (1 - p_i) \;\geq\; 1 - (1 - p_1)^N \quad \text{whenever} \quad p_i \geq p_1, \tag{20}$$

so as long as feedback does not make the model worse, retries with feedback dominate unbatched sampling at equal cost.

With batching ($N \leq N^*$) the comparison changes. If the shared prompt is processed once, as servers that share prefixes across sequences do [11], a call that returns $N$ candidates takes

$$t_N \;=\; \frac{(1-\rho)\, n_{\text{in}}}{r_p} + n_{\text{out}}\, t_{\text{step}}(N) \;\leq\; t\, \frac{M + N\kappa\bar{n}}{M + \kappa\bar{n}}, \tag{21}$$

barely more than a call that returns one. The best policy then combines both: each attempt draws a batch of $N$ candidates, the verifier keeps any that pass, and the failure output of one that did not drives the next attempt. If candidates were independent, attempt $i$ would pass with probability $1-(1-p_i)^N$ at cost $t_N$, and Equation (17) applies with these values. For the 7B model of Section 4.5, four candidates at most 1.08 times the cost of one would raise a first-attempt pass rate of 0.3 to $1-0.7^4 \approx 0.76$, and cut the time per accepted change from $3.3\,t$ to $1.4\,t$. Best-of-$N$ with a verifier is how systems such as AlphaCode [14] use parallel hardware; feedback loops [5, 20] are how a single GPU gets value from sequential calls; batching lets a small GPU have some of both. Independence is optimistic: samples at low temperature are correlated, so batched candidates need a higher temperature than single attempts to be diverse.

### 5.4 Loop detection

Each attempt $i$ produces a reply and, if it fails, a failure output. The harness hashes both after normalizing whitespace: $h_i = H(\text{reply}_i)$ and $f_i = H(\text{failure}_i)$. A step is *stalled* if

$$h_i = h_{i-1} \quad \vee \quad f_i = f_{i-1} \quad \vee \quad h_i = h_{i-2}, \tag{22}$$

covering a repeated reply, an unchanged failure, and an $A \to B \to A$ cycle. On a stall the harness changes the input—targeted feedback naming what was already tried—or escalates the step to a larger local model. With at most $k$ attempts per step and $n_s$ steps, a run makes at most $n_s k$ builder calls, so termination is guaranteed independently of model behavior.

### 5.5 Retrieval fusion

The skeletonizer ranks files by lexical overlap (TF-IDF over identifiers), and an embedding index ranks functions by the cosine similarity $\hat{q}^\top \hat{c}$ between unit-normalized embeddings of the request and of each function. The two rankings are fused by reciprocal rank fusion [7],

$$s(d) \;=\; \sum_{j \in \{\text{lex},\,\text{vec}\}} \frac{1}{k_0 + \text{rank}_j(d)}, \qquad k_0 = 60, \tag{23}$$

which rewards documents that rank highly in either list without requiring their scores to be comparable. Exact identifier matches tend to stay near the top; functions described in other words are added. Algorithm 1 combines these pieces into one step of the loop.

## 6 Comprehensive Systems Bottleneck Analysis

Table 2 lists the principal bottlenecks of the system, the failure each produces, and its mitigation.

## 7 Control Plane

An agent that writes and executes code is a program that runs other programs. Six controls keep it safe, reproducible and able to run unattended.

1. **Sandbox.** Model-written code runs under an explicit network policy, offline by default; network tools are blocked unless allowed.
2. **Review and undo.** Every diff is shown for approval or applied with a one-step undo; unattended runs keep a full trace of each prompt, reply and verdict.
3. **Verification.** Tests, linters and schema checks decide what is kept; any judgement a model makes is re-validated in code.

**Algorithm 1** One step of the harness loop.

**Input:** goal $G$, repository $R_{\text{repo}}$, builder model $m$, attempt cap $k$

$X \leftarrow \textsc{Retrieve}(R_{\text{repo}}, G)$ ▷ skeleton and fused ranking, Eq. (23)
$F \leftarrow \varnothing;\ \mathcal{T} \leftarrow \langle\,\rangle$ ▷ feedback; fingerprints
**for** $i = 1, \ldots, k$ **do**
  $y_i \leftarrow m(\textsc{Prompt}(X, G \,\|\, F))$ ▷ stable prefix $X, G$; tail $F$; Eq. (15)
  $v_i \leftarrow \textsc{Verify}(\textsc{Apply}(y_i))$ ▷ lint, compile, test
  $\textsc{Record}(y_i, v_i)$ ▷ trace; later a training label
  **if** $v_i$ passes **then**
    **return** $\textsc{Accept}(y_i)$
  **end if**
  append $(H(y_i),\, H(v_i.\mathit{failure}))$ to $\mathcal{T}$
  **if** $\textsc{Stalled}(\mathcal{T})$ **then** ▷ Eq. (22)
    $m \leftarrow \textsc{Escalate}(m);\ F \leftarrow \textsc{TargetedFeedback}(\mathcal{T})$
  **else**
    $F \leftarrow v_i.\mathit{failure}$
  **end if**
**end for**
**return** $\textsc{Fail}(\mathcal{T})$ ▷ at most $k$ calls; changes remain undoable

4. **Provenance.** The model, its quantization and any weight edit are recorded with each run; base weights are kept beside orthogonalized ones, and licenses are checked.

5. **Unattended operation.** A backlog runs within a budget of time and items with every gate on; a role whose machine stops answering continues locally.

6. **Self-improvement.** The harness edits its own code in a throwaway copy, gated by its full self-test suite. A candidate $\mathcal{H}'$ replaces the running harness $\mathcal{H}$ only if

$$\text{Gates}(\mathcal{H}') \ \wedge\ \big|\text{checks}(\mathcal{H}')\big| \ \geq\ \big|\text{checks}(\mathcal{H})\big|, \tag{24}$$

   so a self-edit cannot pass by deleting the checks that would have failed it. Promotion is confirmed by a person and can be rolled back in one step.

With an orthogonalized model (Section 2), these controls also take over the caution removed from its weights.

# 8 Implementation

The design is implemented in Architect, a working local harness written against the Python standard library that runs the models already installed in Ollama; the prototype accompanies this draft. Table 3 maps each element to its implementation.

# 9 Proposed Evaluation

Sections 2–5 make predictions that can be tested on consumer hardware, with the harness as the measuring instrument. No results are reported in this draft.

**Hypotheses.**

**Table 2:** Systems bottlenecks and mitigations.

| Subsystem or layer | Bottleneck type | Failure mode | Mitigation |
|---|---|---|---|
| SLM parameter scale | Global architectural horizon | Loses the thread in refactors that span many interdependent files. | Hierarchical multi-pass AST summaries; enforced module boundaries; one module per step. |
| Abliteration vector removal | Calibration and stability | Over-optimism, loss of self-skepticism, and repetition loops on code that is syntactically valid but logically broken. | Hard caps of at most five attempts; state-hash tracking; forced context reset on loop detection; code-decided gates. |
| KV cache and prefill phase | Memory bandwidth and compute | Prompt processing grows with context and agent history, and a full window slows every generated token. | Stable prompt prefixes for cache reuse; FlashAttention; 8-bit KV cache; dynamic context pruning. |
| Memory hierarchy | GPU and RAM capacity | A model that spills into RAM decodes several times slower, one that spills to storage an order of magnitude slower; swaps that miss the page cache take seconds. | Size models and windows from the fit sheet; keep the rotation within RAM; a mixture-of-experts model or a second machine where weights exceed the GPU (Table 1). |
| Best-of-$N$ verification | Compute and execution overhead | Each parallel candidate needs its own KV cache; slow test suites block loop iteration. | Retries with feedback; batched candidates where their caches fit (Section 5.3); fast static linters before the full suite; speculative decoding. |
| Several machines | Availability | A machine that sleeps or drops off the network stalls the roles it serves. | Check each machine before a call; continue on the local machine and say so once. |
| AST and retrieval pipeline | Context resolution | Dynamic imports, runtime reflection and metaprogramming break static AST parsing, leaving symbols out of context. | Hybrid retrieval: static AST symbols fused with vector embeddings, plus runtime traces where available. |

**H1.** *Orthogonalization.* Over-refusal on safe prompts [17] falls to near zero for a 14B and a 32B model; first-attempt pass rates fall most on reasoning-heavy tasks; with the full harness, the pass rate within $k = 5$ attempts stays within a pre-registered margin $\delta = 5$ points of the base model's.

**H2.** *Memory hierarchy.* Decoding speed follows Equation (6) and stays below it, including the drop once a model exceeds RAM and the scaling of mixture-of-experts models with their active weights.

**H3.** *Swaps.* Swaps are warm when Inequality (10) holds and cost about $M/B_{\text{disk}}$ each when it does not.

**H4.** *Feedback and batching.* At an equal number of calls, retries with feedback beat unbatched best-of-$N$ (Equation (20)); the measured $C$ lies below $t/p_1$; batched candidates lower $C$ further where they fit.

**Table 3:** Design elements and their implementation in Architect.

| Element | Implementation | Status |
|---|---|---|
| Orthogonalization | Projection applied with existing abliteration tooling; edited builds are recognized, labelled, and scored against their base on the same code-checked tasks (Section 2.4). | Gate built; projection external |
| Loop detection | Attempt cap; replies and failures fingerprinted; A→B→A cycles detected; stalled steps escalate to a larger local model. | Built |
| Prompts and output | Prefix-stable retry templates, enforced by a self-test; JSON-schema output re-validated in code. | Built |
| Retrieval | AST skeleton plus a function-level vector index fused with TF-IDF by reciprocal rank fusion. | Built |
| Verifier | Syntax, SQL, OpenAPI, placeholder-code and missing-import gates; failure output fed back; related tests first, full suite last. | Built |
| Memory fit and swaps | Fit sheet per model (Equation (5)); windows tuned from measured speed; mixture-of-experts models timed by active weights; steps grouped by model; per-run swap ledger. | Built |
| Several machines | Roles routed to other machines; local fallback when one is unreachable. | Built |
| Unattended operation | Backlog within a budget; self-edits gated by Equation (24) with rollback; battery mode. | Built |
| Cost and placement | $C$ per role and model measured from traces; placement left to the user rather than solved from Equation (18). | Partial |
| Training data | SFT samples and preference pairs from verified runs. | Built |
| Sandbox default | Network policy offline, allowlist or open; the shipped default is open and should be set to offline. | Configuration |
| Throughput | Speculative decoding [13]; batched candidates with a shared prefix; tree-sitter chunking beyond Python. | Proposed |

**H5.** *Prefix reuse.* Prompt processing on retries falls by a factor close to $1 - \rho$ (Equation (15)).

**H6.** *Placement and distribution.* The sign of the change in $J$ when the 32B model plans matches Inequality (19) on measured inputs; on two machines, $J$ approaches the left side of Inequality (12).

**Design.** Base and orthogonalized 14B and 32B models at the same 4-bit quantization; harness ablations (no harness, full, without retrieval, without feedback, without loop detection); an existing open-source harness, Aider,[1] with the same models as a baseline; prefix-stable versus feedback-first prompts; windows from 4k to 32k tokens; model rotations that do and do not fit in RAM; one 6 GB GPU with 16 GB of RAM, and two such machines. Tasks are judged by code: six harness tasks, 100 MBPP problems [3], 30 repository-level fixes with hidden tests, and, for H1, the XSTest safe prompts and 50 GSM8K problems [6] as a reasoning control. Every condition runs three seeds. Metrics are pass@1 [4], pass within $k$, $C$, $J$, decoding and prompt-processing speed, swap times, storage reads and refusal rate.

**Analysis.** Comparisons are paired: McNemar's test for pass/fail, the Wilcoxon signed-rank test for times, and bootstrap 95% confidence intervals over tasks and seeds; H1 is a non-inferiority test against $\delta$. All runs are reported. The main threats are author-chosen tasks, benchmark

[1] `https://aider.chat`

contamination (which inflates absolute rates but not paired comparisons), a single orthogonalization method [23], and hardware-specific timings.

## 10 Research Precedents, Similar Work, and Lessons Learned

**Refusal geometry.** Zou et al. [24] showed that high-level concepts can be read and steered as directions in activation space. Arditi et al. [2] found that across 13 open chat models of up to 72B parameters refusal is mediated by one direction, removable by weight orthogonalization with minimal effect on other capabilities; Wollschläger et al. [21] refine this to several independent directions and concept cones. XSTest [17] measures over-refusal. Labonne [12] reports quality lost to abliteration and largely recovered by DPO [16]; Young [23] compares four tools across 16 models of 7–14B, finding mathematical reasoning the most sensitive, in preliminary results.

**Agentic coding.** Codex [4] and MBPP [3] made test-based pass@$k$ the standard measure; AlphaCode [14] showed best-of-$N$ with a verifier at scale; self-debugging [5] and Reflexion [20] feed execution results back to the model; SWE-agent [22], on SWE-bench [10], showed that the interface between agent and computer matters as much as the model. Open-source harnesses such as Aider already run similar loops on local models; this report adds an explicit cost model for consumer hardware, code-decided gates and orthogonalized models behind a parity gate.

**Inference on limited hardware.** At small batch sizes, decoding is bound by the memory traffic of reading the weights and the KV cache [15], the basis of Equation (6). FlashAttention [8] reduces the memory traffic of attention; grouped-query attention [1] reduces $h_{\mathrm{kv}}$ and therefore $\kappa$ in Equation (4); PagedAttention [11] manages KV-cache memory so that many requests can be batched and share prefixes; speculative decoding [13] uses a small draft model to reduce the number of sequential steps of a large one; sparsely gated mixture-of-experts layers [19] decouple the parameters a model holds from those each token uses; and Dettmers and Zettlemoyer [9] find that for a fixed budget of model bits, 4-bit precision is almost universally optimal for zero-shot accuracy, the basis of the default in Section 4.7. Reciprocal rank fusion [7] combines rankings without calibrating their scores.

**Lessons learned.** Users of local models report that small models fail more often by looping on logical dead ends than by lacking syntax, and that abliteration removes refusals reliably while hedging and disclaimers often persist. Loop caps, state hashing and context pruning are therefore requirements rather than refinements.

## 11 Actionable Engineering Items and Timeline

Durations are indicative; each phase ends when its gate passes on real models, not when the code exists.

**Phase 1.** *Dependable loop (built).* Loop detection, prefix-stable prompts, schema-bound output, hybrid retrieval, code-decided gates, a fit sheet and swap ledger, routing to other machines, an unattended task backlog, and gated self-improvement.

**Phase 2.** *Measure the model (2–4 weeks).* Orthogonalize a 14B and a 32B model and run the evaluation of Section 9 on base and orthogonalized variants; preference tuning from verified runs to heal what orthogonalization costs; per-workstation configuration presets.

**Phase 3.** *Throughput (1–2 months).* Speculative decoding [13]; batched candidates with shared prefixes; tree-sitter chunking beyond Python; independent roles overlapped across machines (Inequality (12)).

**Phase 4.** *Learning from tests (24 GB GPU and above).* Reinforcement learning from pass/fail rewards, for example with GRPO [18], on the harness's own labelled runs.

## 12 Conclusion

Refusal in a small language model turns out to be a low-dimensional, removable feature of its geometry; correctness does not. The architecture described here removes the first in the weights, by orthogonal projection, and enforces the second in code, by a verifier that decides what is kept. On consumer hardware the harness, not the model, is the system: its design follows from a few measurable quantities—where a model's bytes live, what a call costs, and how often its output passes—and each design choice in this report is an inequality or an expectation over those quantities rather than a preference. The memory hierarchy sets the price of a call; the verifier sets how many calls a change takes. The evaluation in Section 9 is the test of these claims.

## Broader Impact and Responsible Use

Orthogonalization (Section 2) is dual-use: a projection that removes over-refusals also removes refusals of harmful requests. We therefore place orthogonalized models behind the control plane of Section 7, evaluate over-refusal on safe prompts rather than compliance with harmful ones, and release no edited weights. Orthogonalized models should be used only for work the operator is authorized to perform, and within the terms of the model's license. The rest of this report concerns the efficiency and reliability of local agentic systems, with the impacts common to coding assistants: faster development, and the risk of accepting plausible but wrong code, which the verifier-first design is meant to reduce.

## Acknowledgments

Drafting, typesetting and source checking were assisted by an AI model (Claude, Anthropic). The author is responsible for the content.

## References

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, 2023. `https://arxiv.org/abs/2305.13245`

[2] A. Arditi, O. Obeso, A. Syed, D. Paleka, N. Panickssery, W. Gurnee, and N. Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. `https://arxiv.org/abs/2406.11717`

[3] J. Austin, A. Odena, M. Nye, et al. Program synthesis with large language models. arXiv:2108.07732, 2021. `https://arxiv.org/abs/2108.07732`

[4] M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021. `https://arxiv.org/abs/2107.03374`

[5] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024. `https://arxiv.org/abs/2304.05128`

[6] K. Cobbe, V. Kosaraju, M. Bavarian, et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021. `https://arxiv.org/abs/2110.14168`

[7] G. V. Cormack, C. L. A. Clarke, and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.

[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2205.14135

[9] T. Dettmers and L. Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning (ICML)*, PMLR 202, 2023. https://arxiv.org/abs/2212.09720

[10] C. E. Jimenez, J. Yang, A. Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. https://arxiv.org/abs/2310.06770

[11] W. Kwon, Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. https://arxiv.org/abs/2309.06180

[12] M. Labonne. Uncensor any LLM with abliteration. Hugging Face blog, 2024. https://huggingface.co/blog/mlabonne/abliteration

[13] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023. https://arxiv.org/abs/2211.17192

[14] Y. Li, D. Choi, J. Chung, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. https://arxiv.org/abs/2203.07814

[15] R. Pope, S. Douglas, A. Chowdhery, et al. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023. https://arxiv.org/abs/2211.05102

[16] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2305.18290

[17] P. Röttger, H. R. Kirk, B. Vidgen, et al. XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of NAACL*, 2024. https://arxiv.org/abs/2308.01263

[18] Z. Shao, P. Wang, Q. Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024. https://arxiv.org/abs/2402.03300

[19] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1701.06538

[20] N. Shinn, F. Cassano, et al. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2303.11366

[21] T. Wollschläger, J. Elstner, S. Geisler, V. Cohen-Addad, S. Günnemann, and J. Gasteiger. The geometry of refusal in large language models: Concept cones and representational independence. In *International Conference on Machine Learning (ICML)*, PMLR 267, 2025. https://arxiv.org/abs/2502.17420

[22] J. Yang, C. E. Jimenez, A. Wettig, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. https://arxiv.org/abs/2405.15793

[23] R. J. Young. Comparative analysis of LLM abliteration methods: A cross-architecture evaluation. arXiv:2512.13655, 2025. https://arxiv.org/abs/2512.13655

[24] A. Zou, L. Phan, S. Chen, et al. Representation engineering: A top-down approach to AI transparency. arXiv:2310.01405, 2023. https://arxiv.org/abs/2310.01405